Missing Ranges (easy)

Problem Statement

You are given two integers, `lower` and `upper`, specifying the inclusive range, and the sorted array of unique integers `nums`, where each element is in between `[lower, upper]`.

We can say the number `n` is missing if `n` is not in the `nums` and falls in the range `[lower, upper].`

Return the `shortest sorted` list of ranges that `covers all missing numbers`. These ranges should fall within the inclusive bounds specified by the lower and upper limit integers.

Examples

Example 1:

- **Input**: `nums = [0, 1, 3, 50, 75]`, `lower = 0`, `upper = 99`
- **Expected Output**: `[[2, 2], [4, 49], [51, 74], [76, 99]]`
- **Justification**: The missing ranges are 2, 4-49, 51-74, and 76-99. Each missing range is represented as a pair in the list, where both numbers are the same for a single missing number.

Example 2:

- **Input**: `nums = [10, 20, 30, 40, 50]`, `lower = 5`, `upper = 55`
- **Expected Output**: `[[5, 9], [11, 19], [21, 29], [31, 39], [41, 49], [51, 55]]`
- **Justification**: The missing ranges are 5-9, 11-19, 21-29, 31-39, 41-49, and 51-55. Each interval is represented as a pair of start and end values.

Example 3:

- **Input**: `nums = [1, 3, 6, 10]`, `lower = 0`, `upper = 12`
- **Expected Output**: `[[0, 0], [2, 2], [4, 5], [7, 9], [11, 12]]`
- **Justification**: The missing ranges are 0, 2, 4-5, 7-9, and 11-12. Each range is represented as a start and end pair.

Try it yourself

Try solving this question here:

Java

```java
import java.util.ArrayList;
import java.util.List;

public class Solution {
    // Method to find missing ranges
    public List<List<Integer>> findMissingRanges(int[] nums, int lower, int upper) {
        List<List<Integer>> result = new ArrayList<>();
        // ToDo: Write Your Code Here.
        return result;
    }
}

```

Test Case | Run Code Result | Stdout

```
[0,1,3,50,75]
0
99
```

Use Example Testcases | Run | Submit

← Previous
Solution: Move Zeroes

Next →
Solution: Missing Ranges

Mark as Completed